Certificates & PKI

chain of trust, SNI, Let's Encrypt and mTLS

Watch the 3-min video

In one line: a certificate binds a name to a public key; browsers trust it by checking signatures up a chain to a root in their trust store, and short-lived certificates with automated renewal (ACME) end surprise expiry outages.

1 What a certificate is

Your HTTPS site works for 90 days. Then one morning every browser shows a red warning: the certificate expired and nobody renewed it. It is one of the most common outages there is.

A certificate is an **X.509** document that **binds a name to a public key**. It contains:

Field	What it holds
Subject Alternative Names (SAN)	The domain names it is valid for, e.g. `shop.example.com` or a wildcard `*.example.com` (one level only)
Public key	The key matching the server's private key, which never leaves the server
Issuer	The certificate authority (CA) that signed it
Validity	`notBefore` and `notAfter` dates
Signature	The issuer's digital signature over all of the above, so no field can be changed

2 The chain of trust

Who signs it? A **certificate authority (CA)**. Your server's **leaf** certificate is signed by an **intermediate CA**. The intermediate is signed by a **root CA**. Root certificates come pre-installed in the operating system and browser: the **trust store**. Roots are kept offline and only sign intermediates, so a compromised intermediate can be revoked without replacing roots on billions of devices.

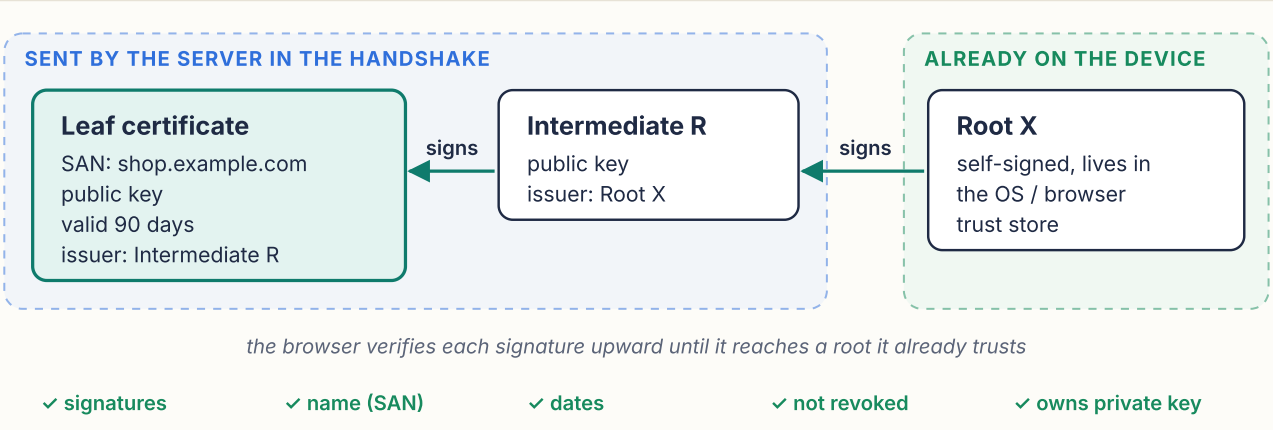


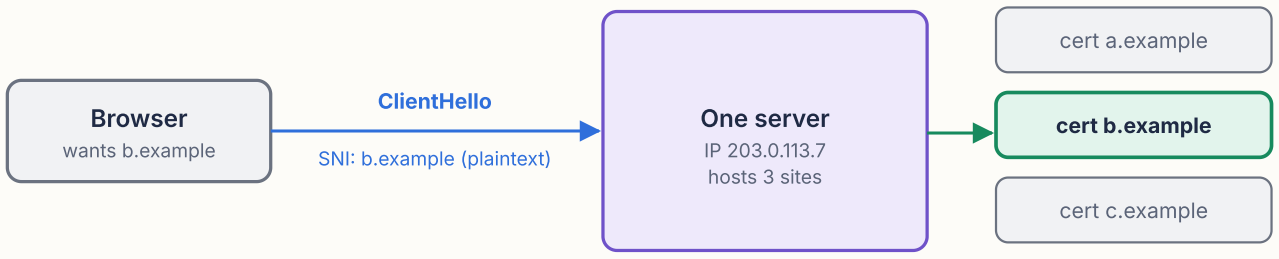
Figure 1. The server sends its leaf plus the intermediate. The root is never sent: the client must already have it. Every check must pass, or the browser shows the warning.

How a browser decides to trust a certificate:

1. Verify each signature up the chain until it reaches a root in its trust store.
2. Check that the site's hostname matches a name in the SAN.
3. Check that today is between the validity dates.
4. Check that the certificate has not been **revoked** (via CRLs, OCSP or the browser's own revocation lists).
5. In the handshake, the server **signs** with the matching private key (CertificateVerify), proving it owns the certificate and did not just copy it.

3 SNI: many sites, one IP address

One server or load balancer often hosts many domains on a single IP address. Which certificate should it send? **Server Name Indication (SNI)**: the client puts the hostname in its ClientHello, so the server picks the right certificate *before* encryption starts. Because it comes first, SNI is sent in plaintext (Encrypted Client Hello, ECH, is the newer fix).



the hostname arrives before encryption starts, so the server can choose the right certificate

Figure 2. SNI lets one IP serve many HTTPS sites, each with its own certificate. Without SNI the server would have to guess, and usually sends a default certificate with the wrong name.

4 Rotation: Let's Encrypt, ACME and cert-manager

Let's Encrypt issues free certificates that last **90 days**, using the **ACME** protocol. To get one you prove you control the domain with a **challenge**:

- **HTTP-01:** serve a token file at `http://your-domain/.well-known/acme-challenge/…` on port 80.
- **DNS-01:** create a TXT record at `_acme-challenge.your-domain`. Needed for wildcard certificates, and works for servers not reachable from the internet.

In Kubernetes, **cert-manager** automates it: it requests certificates, stores them as Kubernetes **Secrets** that ingresses use, and renews them well before they expire (by default when a third of the lifetime is left). Short lifetimes plus automation mean no surprise expiries, and a leaked key is useful for weeks, not years. The industry is pushing lifetimes even shorter, so manual renewal is no longer realistic.



renew early: 30 days leaves time for several failed attempts and for someone to notice the alert

Figure 3. A 90-day certificate renewed at day 60. Alert if any certificate has fewer than ~14 days left: that means automation has been failing for two weeks.

5 Mutual TLS (mTLS)

Normally only the server shows a certificate. With **mutual TLS**, the client presents one too, and the server checks it against a trusted CA. This is ideal for **service-to-service authentication**: the payments service accepts calls only from services holding a valid internal certificate, issued by your own private CA. Service meshes like **Istio** and **Linkerd** run a sidecar proxy next to each service that issues, presents and rotates these certificates automatically (often every 24 hours), so application code does not change.

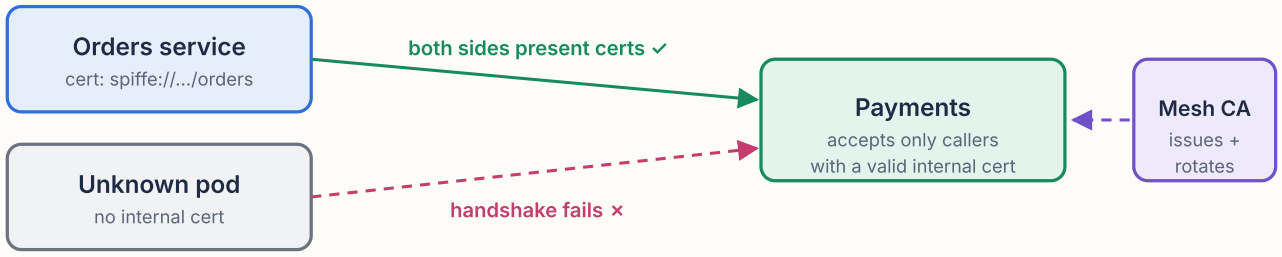


Figure 4. With mTLS the payments service knows which service is calling, from its certificate, before any request is read. A pod without a valid certificate cannot even finish the handshake.

WATCH OUT: CERTIFICATE PINNING

Some mobile apps **pin**: they hard-code which certificate or public key they will trust. If you rotate to a new key the app doesn't know, every installed copy stops connecting until users update. If you pin, pin a backup key too and plan rotation, or pin the CA's intermediate instead of your leaf.

WORKED EXAMPLE

You run 40 domains. With manual 1-year certificates that is about 40 renewals a year, each one a chance to forget. With ACME and 90-day certificates renewed at 30 days left, that is about 160 renewals a year, all automatic, each with a 30-day window to retry. Monitor `notAfter` for every endpoint, and page when any is under 14 days.

COMMON MISTAKES

- Renewing by hand and relying on someone's calendar.
- Serving the leaf without the intermediate: works in browsers that cached it, fails in curl, mobile apps and containers.
- Relying on the certificate's CN instead of SAN, or expecting `*.example.com` to cover `a.b.example.com`.
- Pinning a leaf key in a mobile app with no backup pin.

6 Key terms

X.509: the standard certificate format

CA: certificate authority, signs certificates

Leaf / intermediate / root: the three links of a chain

Trust store: roots pre-installed in OS and browser

SAN: the names a certificate is valid for

SNI: hostname in the ClientHello

ACME: protocol for automated issuance and renewal

cert-manager: automates ACME in Kubernetes

mTLS: both client and server present certificates

7 Interview questions

Q1. How does a browser trust a certificate? BASIC

It verifies the chain of signatures from the leaf, through intermediates, to a root CA in its trust store. It checks the hostname against the SAN, the validity dates and revocation. And in the handshake the server signs with the matching private key.

Q2. What is SNI and why is it needed? BASIC

Server Name Indication: the client sends the hostname in the ClientHello, so a server hosting many domains on one IP can present the matching certificate before encryption starts.

Q3. What is mTLS used for? BASIC

Mutual authentication, mostly between services: both sides present certificates, so the server knows exactly which service is calling. Meshes like Istio and Linkerd issue and rotate them automatically.

Q4. Your site went down because a certificate expired. How do you prevent it from happening again? SCENARIO

Automate with ACME (certbot, cert-manager, or managed certificates on the load balancer), renewing at 30 days left. Monitor notAfter for every endpoint and alert below 14 days, so failing automation is caught with two weeks to spare.

Q5. You need a certificate for *.internal.example.com on servers not reachable from the internet. Which ACME challenge? SCENARIO

DNS-01: create a TXT record at _acme-challenge.internal.example.com. HTTP-01 needs port 80 reachable from the internet and can't issue wildcards.

Q6. curl says 'unable to verify the first certificate' but the browser shows the padlock. Why? SCENARIO

The server isn't sending its intermediate certificate. The browser had cached it or fetched it on its own; curl and most libraries don't. Configure the server to send the full chain (leaf + intermediate).

Q7. Why do CAs use intermediates instead of signing leaves with the root? TRICKY

The root key stays offline in an HSM, so it's hard to steal. If an intermediate is compromised it can be revoked and replaced, without updating trust stores on billions of devices, which takes years.

Q8. Why are short certificate lifetimes considered more secure? TRICKY

Revocation checking is unreliable (browsers often soft-fail), so a stolen key stays usable until expiry. With 90-day or shorter certificates, the damage window is weeks, not years. It also forces automation, which removes human error.

Next: SD-B-013 · HTTP/1.1 vs HTTP/2 vs HTTP/3 backendroadmap.com